PYTHON FOR ACCOUNTING

Program 1:

Write a Program to read data from data file and show Data File Handling related functions utility in python.

Aim:

To Write a Program to read data from data file using data file handling functions.

Algorithm:

Step 1 : Click on start → all programs → Python

Step 2 : In python shell → click File → new file → create a file and save the file.

Step 3 : click file → new file → write coding

Step 4 : Use built-in Open ( ) function to open a file in python.

Step 5 : By using read () function, a content of the file can be read.

Step 6 : Write ( ) function helps to erase all old content in file and to create a new content.

Step 7 : Append ( ) function helps to add the content at the end of the file.

Step 8 : Save the file using .py extension and run the program.

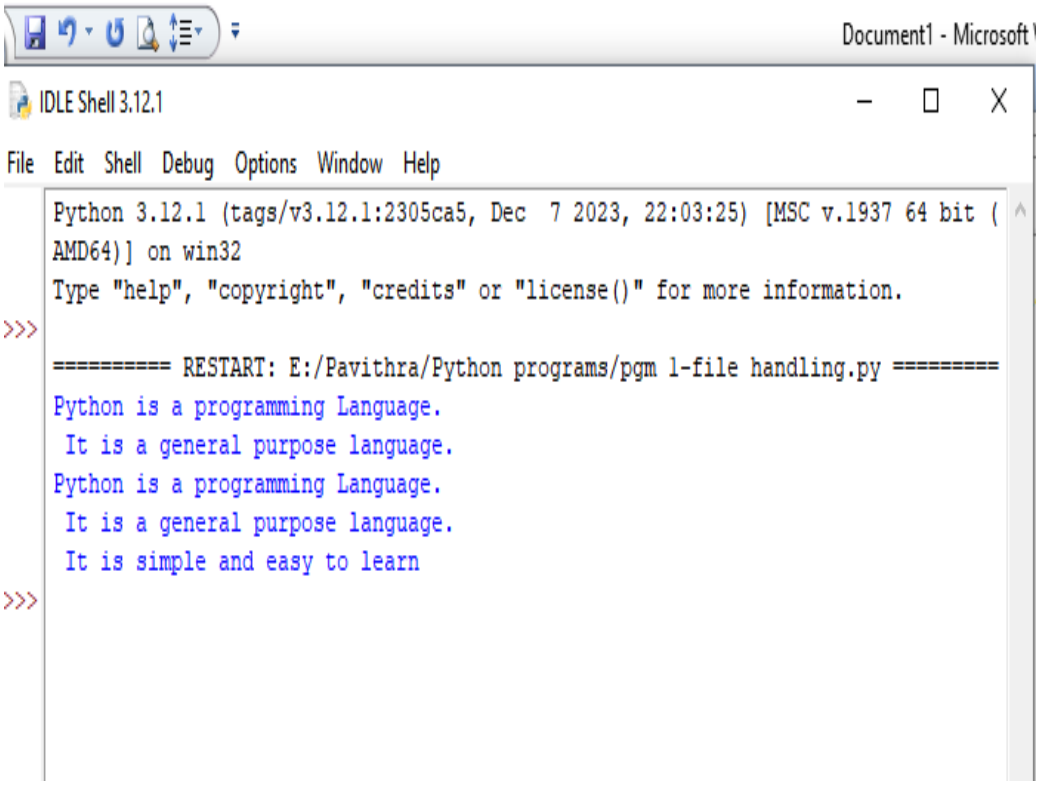
Coding:

```
file=open("Introduction.txt","w")
file.write("Python is a programming Language. \n It is a general purpose language.")
file=open("Introduction.txt","r")
print(file.read())
file=open("Introduction.txt","a")
file.write("It is simple and easy to learn")
file.close()
file=open("Introduction.txt","r")
print(file.read())
```

Result :

The program executed successfully.

Output:



<u>Program 2:</u>

Perform all the operations with reference to table 'Employee' through MySQL-Python connectivity.

a. Fetching all the records from EMP table having salary more than 70000.

b. Deleting record(s) from table using DELETE

c. Update Record(s) to the table using UPDATE

Aim:

To write a program to perform the Fetch data, Update and delete functions from employee database.

Algorithm:

Step 1 : Click on start → all programs → Python

Step 2 : Type pip install pandas (to read the data) and pip install openpyxl(to open the data) in command prompt to install the python libraries using pip.

Step 3 : Create an employee database and save the file.

Step 4 : To fetch the data of an employee whose salary is greater than 70000 use panda library and get data from df (dataframe).

Step 5 : To delete and update data open the file using openpyxl and delete the particular data or row or update the data using openpyxl.load_workbook ("filename.xlsx")

Step 6 : Save the file using .py extension in the same directory were the excel file is saved and run the program.

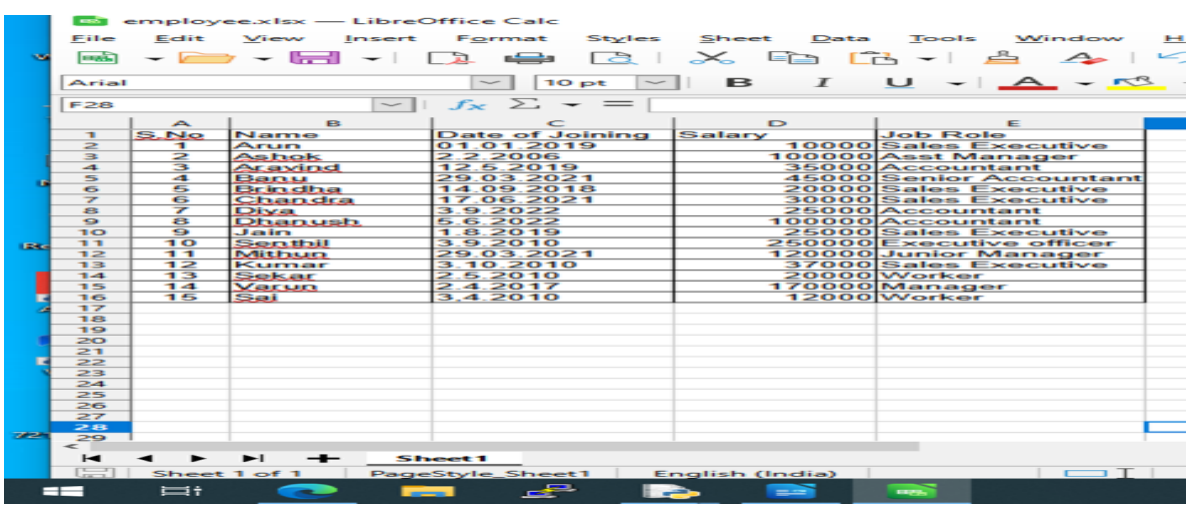
Result :

The program executed successfully.

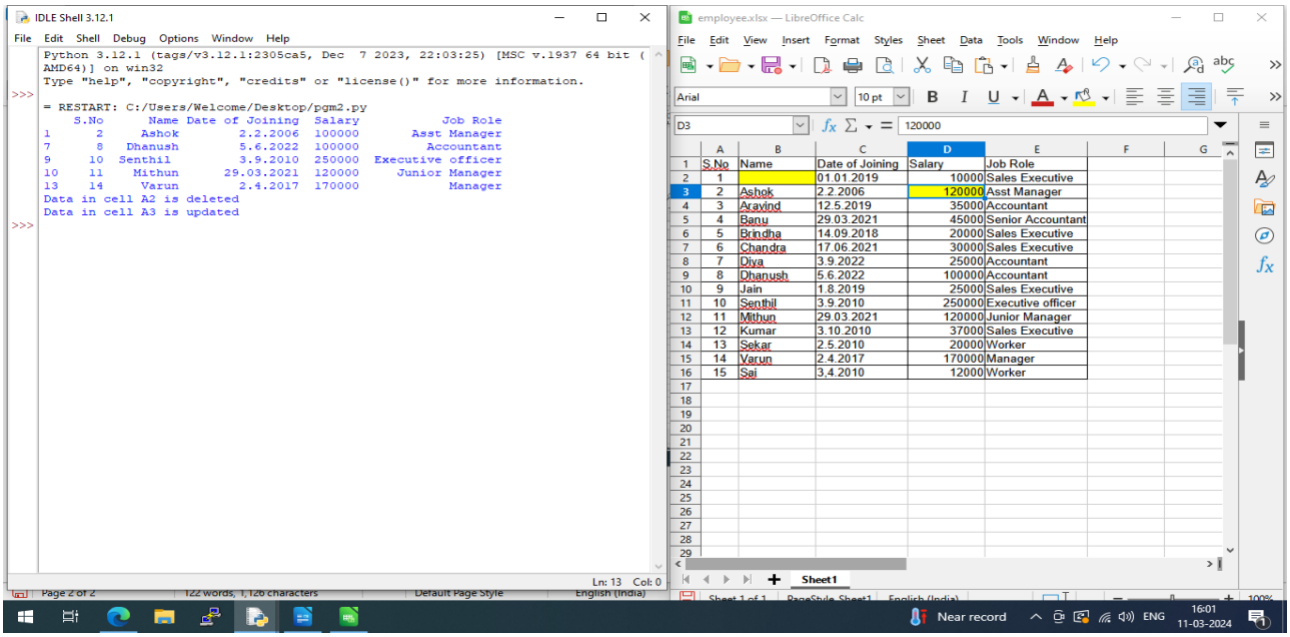
Coding

```
#for fetching employee data with criteria
import pandas as pd
df=pd.read_excel("employee.xlsx")
result=df[df['Salary']>70000]
print(result)
#for deleting a record/data
import openpyxl
workbook=openpyxl.load_workbook("employee.xlsx")
worksheet=workbook.active
worksheet['B2']= None
workbook.save("employee.xlsx")
print("Data in cell A2 is deleted")
#for updating a record
import openpyxl
workbook=openpyxl.load_workbook("employee.xlsx")
worksheet=workbook.active
worksheet['D3']= 120000
workbook.save("employee.xlsx")
print("Data in cell A3 is updated")
```

Output



S.No	Name	Date of Joining	Salary	Job Role
1	Arun	01.01.2019	10000	Sales Executive
2	Ashok	2.2.2006	100000	Asst Manager
3	Aravind	12.5.2019	35000	Accountant
4	Banu	29.03.2021	45000	Senior Accountant
5	Brindha	14.09.2018	20000	Sales Executive
6	Chandra	17.06.2021	30000	Sales Executive
7	Diya	3.9.2022	25000	Accountant
8	Dhanush	5.6.2022	10000	Accountant
9	Jain	1.8.2019	25000	Sales Executive
10	Senthil	3.9.2010	250000	Executive officer
11	Mithun	29.03.2021	120000	Junior Manager
12	Kumar	3.10.2010	37000	Sales Executive
13	Sekar	2.5.2010	20000	Worker
14	Varun	2.4.2017	170000	Manager
15	Sai	3.4.2010	12000	Worker



Program 3

EMI chart for a loan in Python

Aim:

Write a program to create an EMI Chart for a loan.

Algorithm:

Step 1 : Click on start → all programs → Python

Step 2 : In python shell → click File → new file → write coding

Step 3 : Use Input ( ) function to get inputs for Principal, Rate of Interest and Number of years from user.

Step 4 : Calculate EMI using the following formula

EMI = [P X R X (1+R) ^ N] / [(1+R) ^ N-1]

Where p is Principal, R –Monthly rate of Interest N= Loan Tenure in months

Step 5 : By using round ( ) function decimal values can be reduced.

Step 6 : Calculate the remaining principal amount and interest after every installment.

Step 7 : Use Print ( ) statement to print the EMI chart.

Step 8 : Save the file using .py extension and run the program.

Result :

The program executed successfully.

Coding

```
#here p is principal,r is rate interest,t is tenure
p=float(input("enter principal amount: "))
```

```
r=float(input("enter the rate of interest per annum in %: " ))
t=int(input("enter the no of years: "))
#nit is no of installments for year
#mri is monthly rate of interest
nit=t*12
mri=r/(12*100)
#here calcualting the monthly emi
emi=p*mri*((1+mri)**nit)/(((1+mri)**nit)-1)
#\n for new line
print("\nMonthly emi is ",emi)
#here assigning the prinicipal to the remaining principal amount at first installment
rpy=p
print("**************************")
print("\nEMI schedule is ")
print("**************************")
#this print statement prints one new empty line
print()
for i in range(1,nit+1):
    minterest=round(rpy*mri,2)
    principal_pay=round(emi-minterest,2)
    rpy=round(rpy-principal_pay,2)
    print("no of isntallment",i)
    print("monthly interesr",minterest)
    print("principal payment is ",principal_pay)
    print("remaining principal payment",rpy)
    print()
"""
above we used round(variable,value) function it is python inbuilt
function to round off the float values to 2 decimals only"""
```

Output:

```
enter principal amount: 100000
enter the rate of interest per annum in %: 10
enter the no of years: 1
```

Monthly emi is 8791.58872300099
****

EMI schedule is
****
no of isntallment 1
monthly interesr 833.33
principal payment is 7958.26
remaining principal payment 92041.74
no of isntallment 2
monthly interesr 767.01
principal payment is 8024.58
remaining principal payment 84017.16
no of isntallment 3
monthly interesr 700.14
principal payment is 8091.45
remaining principal payment 75925.71
no of isntallment 4
monthly interesr 632.71
principal payment is 8158.88
remaining principal payment 67766.83
no of isntallment 5
monthly interesr 564.72
principal payment is 8226.87
remaining principal payment 59539.96
no of isntallment 6
monthly interesr 496.17
principal payment is 8295.42
remaining principal payment 51244.54
no of isntallment 7
monthly interesr 427.04
principal payment is 8364.55
remaining principal payment 42879.99

no of isntallment 8

monthly interesr 357.33

principal payment is 8434.26

remaining principal payment 34445.73

no of isntallment 9

monthly interesr 287.05

principal payment is 8504.54

remaining principal payment 25941.19

no of isntallment 10

monthly interesr 216.18

principal payment is 8575.41

remaining principal payment 17365.78

no of isntallment 11

monthly interesr 144.71

principal payment is 8646.88

remaining principal payment 8718.9

no of isntallment 12

monthly interesr 72.66

principal payment is 8718.93

remaining principal payment -0.03

Program 4:

Create a dashboard based on Excel data in Python

Aim:

Write a program to create a dashboard from excel data.

Algorithm:

Step 1 : Click on start → all programs → Python

Step 2 : Type pip install pandas, pip install matplotlib, pip install plotly in command prompt to install the python libraries to create a dashboard.

Step 3 : Create an excel data and save the file.

Step 4 : Read the data from the excel and create a bar chart using plt.bar.

Step 5 : Specify the figure Size, Specify the name for X axis, Y axis and chart title.

Step 6 : Save the file using .py extension in the same directory were the excel file is saved and run the program.

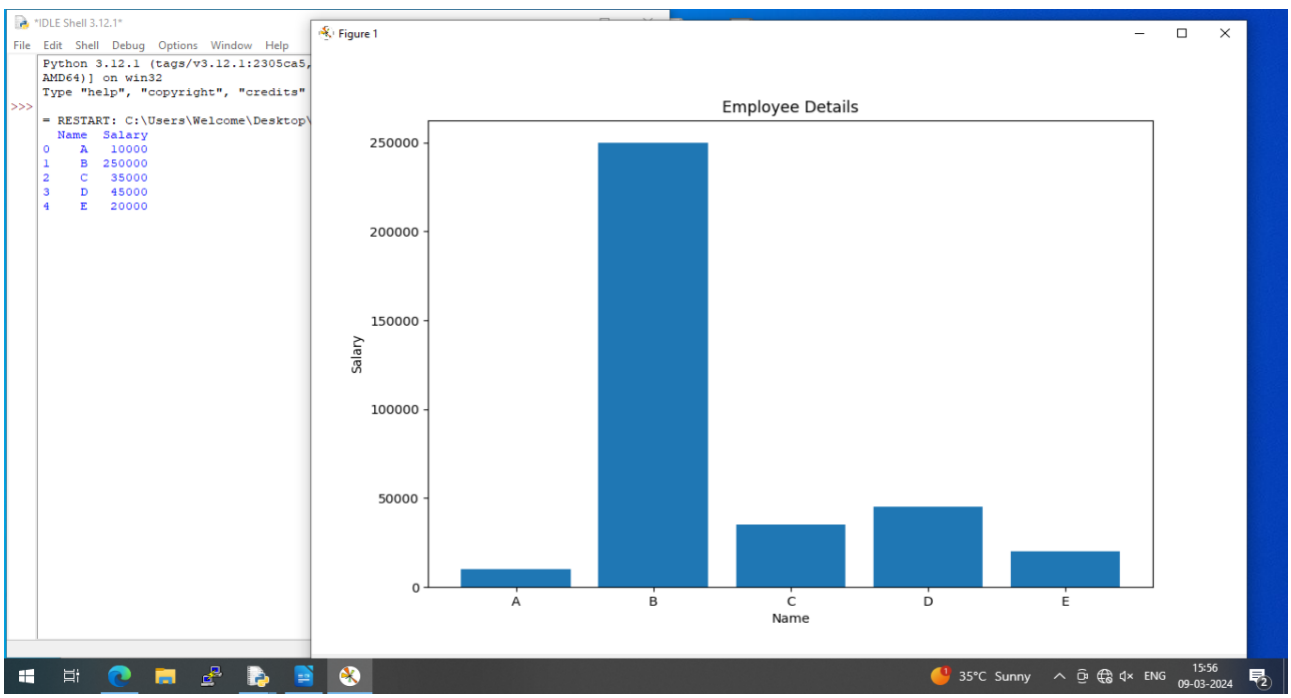
Result :

The program executed successfully.

Coding:

```
import pandas as pd
import matplotlib.pyplot as plt
#df stands for excel dataframe
df=pd.read_excel("employee.xlsx")
print(df.head())
#for creating a bar diagram
plt.figure(figsize=(10,6))
plt.bar(df['Name'], df['Salary'])
plt.xlabel('Name')
plt.ylabel('Salary')
plt.title('Employee Details')
plt.show()
```

Output:



Program 5

Create a simple stock portfolio tracker that calculates the total value of a portfolio based on the number of shares and current stock prices.

Aim:

To write a program to create stock portfolio using number of shares and current stock price..

Algorithm:

Step 1 : Click on start → all programs → Python

Step 2 : In python shell → click File → new file → write coding

Step 3 : Use **def** keyword to define a function, it has set of codes that executes when it is called.

Step 4 : Use Input ( ) function to get inputs for Company Name, Number of shares and current stock price per share from the user.

Step 5 : Calculate the value of shares by multiplying number of shares with share price

Step 6 : Add the value of all stocks to print the "portfolio summary"

Step 7 : Save the file using .py extension and run the program.

Result :

The program executed successfully.

Coding

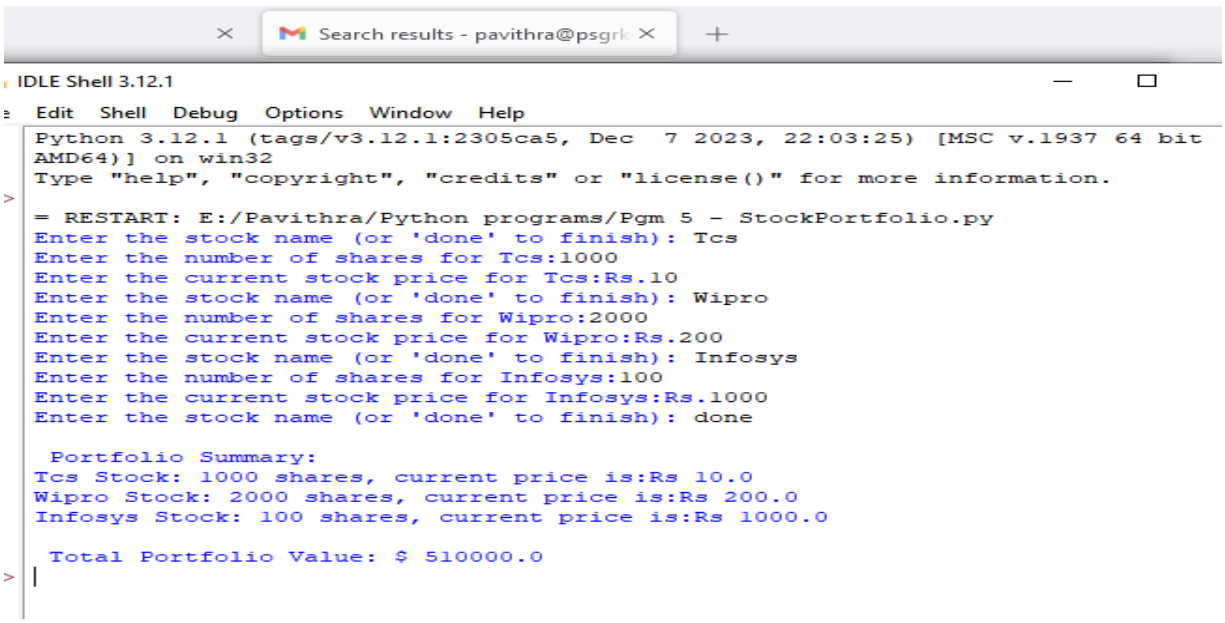
```
def calculate_portfolio_value(portfolio):
    total_value = 0
    for stock in portfolio:
        shares = stock['shares']
        stock_price = stock['stock_price']
        total_value += shares * stock_price
    return total_value
def main():
    portfolio = []
    while True:
        stock_name = input("Enter the stock name (or 'done' to finish): ")
        if stock_name == 'done':
            break
        shares = int(input("Enter the number of shares for {}:".format(stock_name)))
        stock_price = float(input("Enter the current stock price for {}:Rs.".format(stock_name)))
```

```
        stock = {'name': stock_name, 'shares': shares, 'stock_price': stock_price}
        portfolio.append(stock)
        portfolio_value = calculate_portfolio_value(portfolio)
    print("\n Portfolio Summary:")
    for stock in portfolio:
        print(stock['name'],"Stock:",stock['shares'],"shares,", "current price
is:Rs",stock['stock_price'])


    print("\n Total Portfolio Value: $",portfolio_value)


if __name__ == "__main__":
    main()
```

Output:



Program 6

Create a simple program to help users track their monthly income, expenses, and savings, and provide insights into their financial habits.

Aim:

To write a program to display the users monthly income, expenses, and savings.

Algorithm:

Step 1 : Click on start → all programs → Python

Step 2 : In python shell → click File → new file → write coding

Step 3 : Get input from user for monthly income and monthly expenses by using Input ( ) function.

Step 4 : Calculate savings with the following formula savings = income – expenses.

Step 5 : Use print ( ) statement to display the financial summary.

Step 6 : Use if, elif and else statement to check whether the savings is positive, negative or zero.

Step 7 : Save the file using .py extension and run the program

Result :

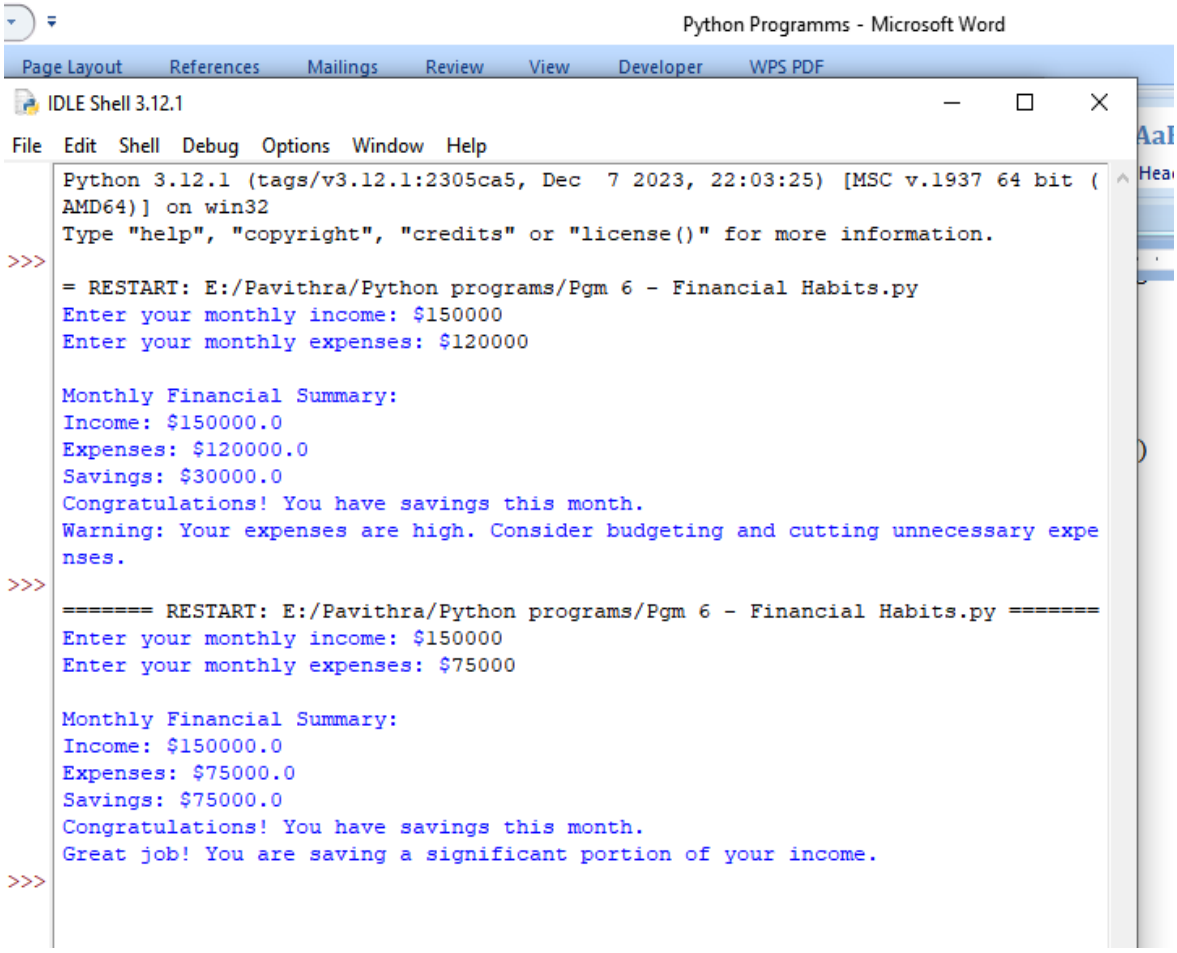
The program executed successfully.

Coding

```
def track_finances():
    # Get user input for income, expenses, and savings
    income = float(input("Enter your monthly income: $"))
    expenses = float(input("Enter your monthly expenses: $"))
    # Calculate savings
    savings = income - expenses
    # Display the results
    print("\n Monthly Financial Summary:")
    print("Income:", income)
    print("Expenses: $", expenses)
    print(f"Savings: $", savings)
    # Check if savings are positive, negative, or zero
    if savings > 0:
        print("Congratulations! You have savings this month.")
    elif savings < 0:
        print("Oops! Your expenses exceed your income. Consider cutting expenses.")
    else:
        print("Your income equals your expenses. Keep an eye on your budget.")
    # Provide insights into financial habits
    if expenses > income * 0.7:
        print("Warning: Your expenses are high. Consider budgeting and cutting unnecessary
expenses.")
    elif savings > income * 0.2:
        print("Great job! You are saving a significant portion of your income.")
    else:
```

```
    print("Consider reviewing your budget to find opportunities for saving.")
# Run the program
track_finances()
```

Output



<u>Program 7</u>

Write a program that uses historical stock price data to implement a simple algorithmic trading strategy, such as a moving average crossover.

Aim:

To write a program to calculate moving average for trading strategy using historical stock price.

Algorithm:

Step 1 : Click on start → all programs → Python

Step 2 : In python shell → click File → new file → write coding

Step 3 : Create a function using Def with stock price and days size.

Step 4 : Moving average can be calculated with the following formula

Moving Average = Sum of Prices for 'N' Days / days size

Step 5 : Use [ ] (Square Brackets) to list the closing share price and mention the number of days (N) for calculate the moving average.

Step 6 : To display the results use Print ( ) function.

Step 7 : Save the file using .py extension and run the program

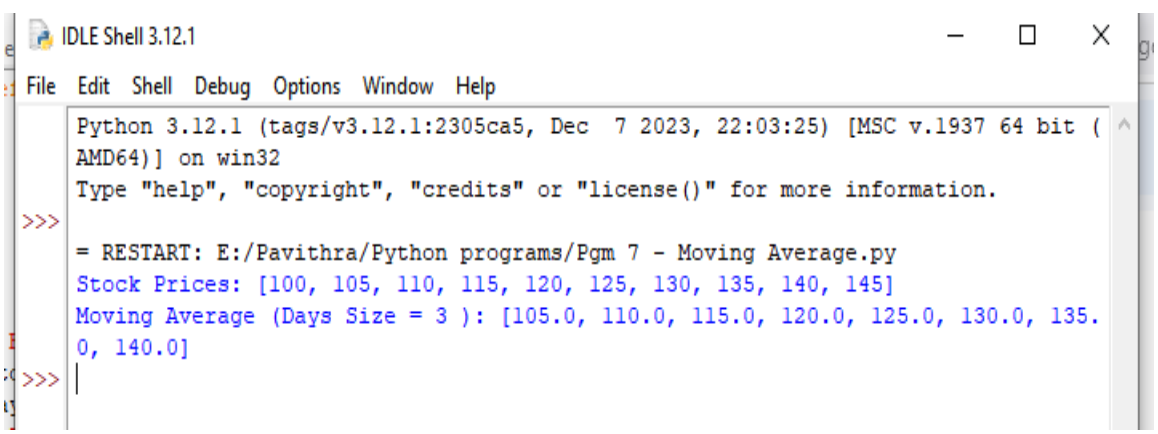
Result :

The program executed successfully.

Coding

```
def moving_average(stock_prices, days_size):
    moving_averages = []
    for i in range(len(stock_prices) - days_size + 1):
        day = stock_prices[i:i + days_size]
        average = sum(day) / days_size
        moving_averages.append(average)
    return moving_averages
# Example stock price data
stock_prices = [100, 105, 110, 115, 120, 125, 130, 135, 140, 145]
days_size = 3
# Adjust the window size as needed
# Calculate moving average
averages = moving_average(stock_prices, days_size)
# Display the results
print("Stock Prices:", stock_prices)
print("Moving Average (Days Size =", days_size, "):", averages)
```

Output



Program 8

Build a data visualization tool to create interactive financial charts and plots using libraries

like Matplotlib and Plotly.

Aim:

Write a program to to create interactive financial charts and plots using libraries like Matplotlib and Plotly.

Algorithm:

Step 1 : Click on start → all programs → Python

Step 2 : Type pip install pandas, pip install matplotlib, pip install plotly in command prompt to install the python libraries to create a dashboard.

Step 3 : Create a financial data – Stock price in idle shell using key attribute {'Date': 'Closing Price'}

Step 4 : Read the data using df ie dataframe create a plot using plt.plot( ).

Step 5 : Specify the name for X axis, Y axis and chart title.

Step 6 : Save the file using .py extension and run the program.

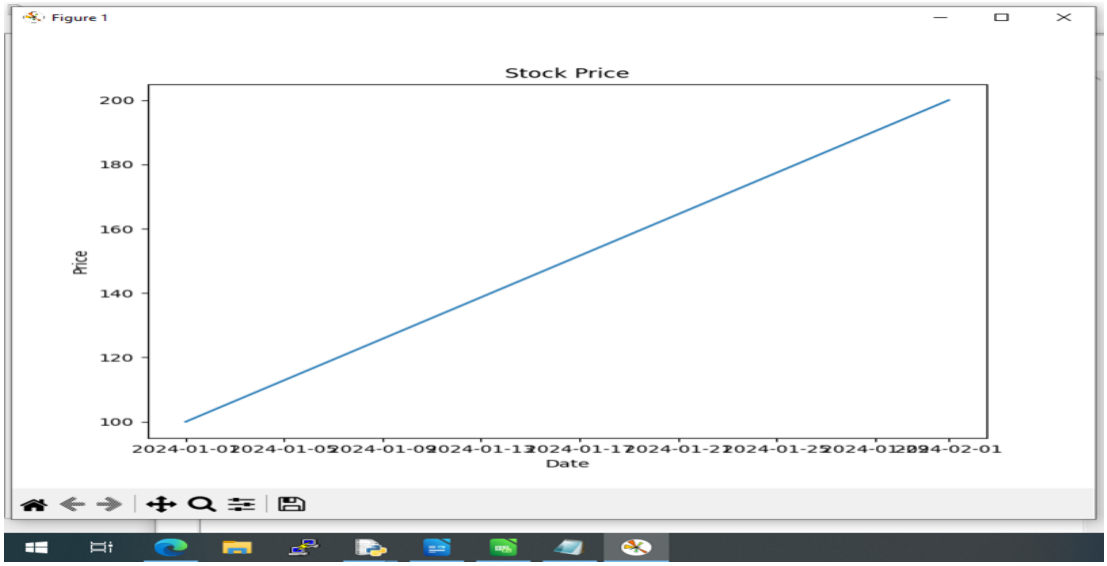
Result :

The program executed successfully.

Coding:

```
import pandas as pd
import matplotlib.pyplot as plt
data={'Date':['01.01.2024','02.01.2024'],'Price':[100,200]}
df=pd.DataFrame(data)
df['Date']=pd.to_datetime(df['Date'])
plt.plot(df['Date'],df['Price'])
plt.title('Stock Price')
plt.xlabel('Date')
plt.ylabel('Price')
plt.show()
```

Output



Program 9

Building an Automated Data Extraction Solution (from e-governance sites) with Python.

Aim:

Write a program to Automated Data Extraction Solution (from e-governance sites).

Algorithm:

Step 1 : Click on start → all programs → Python

Step 2 : In python shell → click File → new file → write coding

Step 3 : Type pip install requests beautifulsoup4 in command prompt. This library helps to download the content from website.

Step 4: Type the website link address using url="website link"

Step 5 : Get the data using requests.get(url) and check the status code(it should be 200)

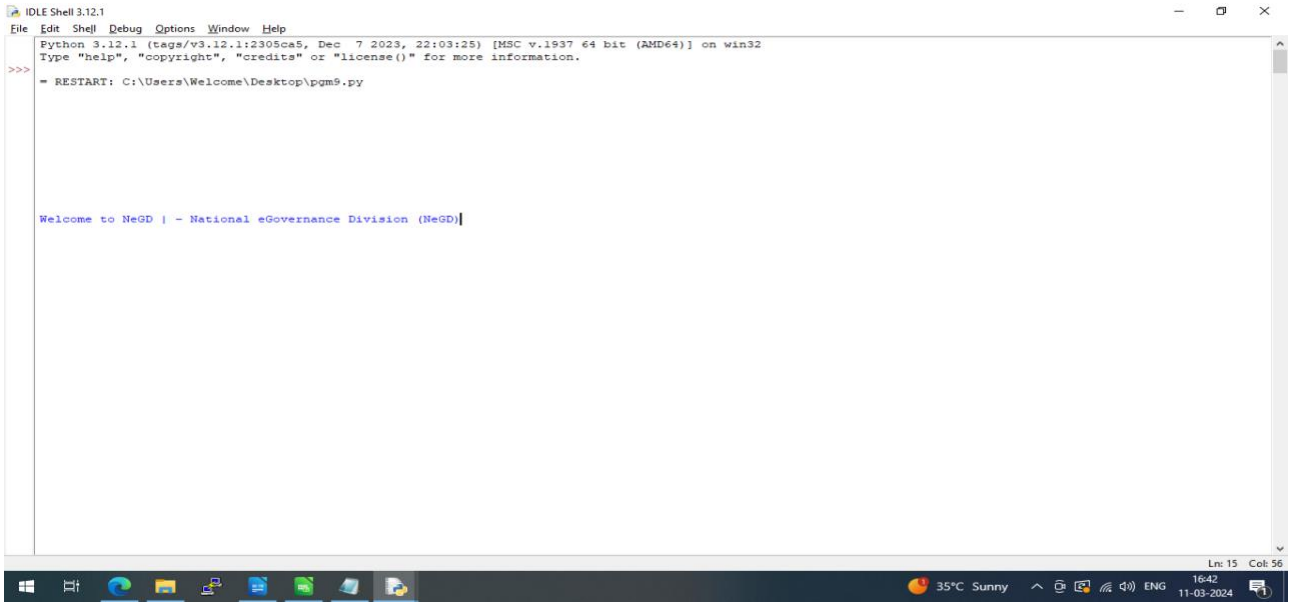
Step 6 : Get all data from website using BeautifulSoup and print all data.

Step 7 : Save the file using .py extension and run the program

Result :

The program executed successfully.

Coding:

```
import requests
from bs4 import BeautifulSoup
url='https://negd.gov.in/'
response=requests.get(url)
if response.status_code == 200:
    soup=BeautifulSoup(response.text,'html.parser')
    all_data=soup.get_text()
    print(all_data)
```

```
else:
    print("Failed to fetch the webpage")
```